

CAMP: Graph-Aware Prefetching and Pinning for LLM Inference over Host-Mediated CXL Memory

Quang-Vinh Dang^{a,*}

^a*British University Vietnam, Hung Yen, Vietnam*

Abstract

Large language models whose weights exceed GPU memory can be served by keeping part of the weights in CXL-attached memory and streaming them into HBM. A CXL Type-3 expander is not addressable by the GPU, so every transfer is a host-initiated DMA limited by the slower of the CXL device and the PCIe link, which published prototypes put at roughly 10 to 20 GB/s. We present CAMP, a runtime policy that exploits the static layer order of inference through three components: an online-calibrated roofline cost model that needs no offline profiling, a work-conserving prefetcher with next-use admission, and stall-cost-aware pinning (SCP), which scores candidate sets of resident layers with a pipeline planner and verifies them in a dry run. We evaluate with an event-driven simulator whose copy path, compute model and noise follow measured data; its pipeline logic is checked against a real GPU on 30 streaming configurations (7.6% mean error overall, 4.0% when link-bound). On Llama-2, Llama-3 and Gemma-2 models, for prefill, decode and mixed batches, CAMP stays within 6% of a lower bound and is up to 1.47 times faster than the best static baseline and 2.9 times faster than demand paging. The gain comes from where layers are pinned, not how often they are reused: contiguous pinning idles the link, and a frequency rule can lose to random pinning. Streaming from CXL is nearly free only above about 16k tokens per iteration; small-batch decode is 20 to 80 times slower than resident weights.

Keywords: Compute Express Link, memory disaggregation, LLM inference, prefetching, weight offloading, discrete-event simulation

*Corresponding author. ORCID: 0000-0002-3877-8024

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

1. Introduction

The weights of current large language models no longer fit in the memory of a single accelerator: Llama-3-70B needs 141 GB in FP16 and Llama-3.1-405B needs 812 GB, against 80 GB of HBM on a high-end GPU. The usual remedies are to shard the model over several GPUs, which multiplies the hardware bill, or to offload part of it to host DRAM, SSD or a remote tier and stream it back on demand [1, 2]. Compute Express Link (CXL) memory expanders add a new option: byte-addressable DDR capacity attached to the host over PCIe-class links, at lower cost per gigabyte than HBM and without the capacity ceiling of the DIMM slots [3, 4]. The catch is bandwidth. Prototype Type-3 devices sustain on the order of 10–20 GB/s towards a GPU [5, 6], two orders of magnitude below HBM.

Two facts shape the problem and are easy to overlook when CXL memory is treated as a directly attached GPU memory tier. First, a CXL Type-3 expander is a CXL.mem endpoint behind the host root complex; a GPU cannot address it directly. Weights reach HBM through a host-initiated `cudaMemcpyAsync` from page-locked, CXL-backed host memory, so the achievable rate is set by the slower of the CXL device and the PCIe link, and every copy carries software and synchronisation costs [5]. Second, the access order of transformer inference is static, but the compute available to hide a transfer is not: it scales with the number of tokens processed per iteration. Streaming a layer of p parameters costs $2p$ bytes of link traffic and enables $2pT$ FLOPs for T tokens, so a link of 18 GB/s hides the transfer only when T is in the tens of thousands (Section 3). Prefill of long prompts and very large batches are in that regime; interactive decode is not.

This paper studies weight streaming over a host-mediated CXL path and proposes CAMP (Content-Aware Memory Prefetching), a runtime policy that decides *when* to fetch layers and *which* layers stay resident in HBM. The policy is deliberately small, because the access order is known: it combines (i) an online-calibrated roofline cost model that predicts per-layer compute time without offline profiling, (ii) a work-conserving prefetcher that admits a fetch only if it displaces something needed later, and (iii) *stall-cost-aware pinning* (SCP), which chooses the resident set by minimising a predicted iteration time rather than by ranking layers by reuse frequency. We are explicit about what is and is not new. Hot/cold partitioning by frequency [7], static placement with overlapped I/O [1] and layer-wise DMA prefetching from CXL-attached memory [6] exist; we implement them as baselines. What

we add is a quantified account of when streaming from CXL pays off, a pin-selection rule that accounts for pipeline placement, and a validation of the simulator’s pipeline machinery on a real GPU.

Contributions.

1. **Path model and regime analysis (Sections 3, 6.1).** A host-mediated transfer model with explicit initiator, bandwidth bound, per-copy and synchronisation costs and a staged (bounce-buffer) variant; a closed-form arithmetic-intensity threshold; and a regime map showing that CAMP-managed streaming is within $1.14\times$ of resident weights only above roughly 16k tokens per iteration (18 GB/s), while decode with at most 256 tokens per iteration is $66\text{--}79\times$ slower.
2. **The CAMP policy (Section 4).** A runtime cost model that is calibrated online from observed layer durations (23% generic roofline error becomes 0.5–2.7% after one probe iteration); a work-conserving prefetcher whose decisions do not depend on the compute estimate; and SCP, a plan-and-verify pin selection whose objective, constraints and relation to the 0–1 knapsack are stated exactly.
3. **Where pinning gains come from (Sections 6.3, 6.4).** In link-bound decode every sensible pin rule is within 3% of the others; the benefit of SCP appears when compute and transfer are comparable, where contiguous (prefix or frequency-ranked) pinning leaves the link idle 12–21% of the time and can be slower than random pinning. LFU is indistinguishable from LRU on cyclic uniform traces, so a frequency-based eviction policy provides no protection there.
4. **Evaluation methodology (Sections 5, 6).** An event-driven simulator whose pipeline logic is validated against a real GPU on 30 streaming configurations, an implementation-independent lower bound, ten seeds with common random numbers, and evaluation on Llama-2/3 and Gemma-2 configurations from 9B to 405B parameters in prefill, decode and mixed-batch iterations. CAMP is within 6% of the lower bound in all 16 main configurations and up to $1.47\times$ faster than the best static baseline.

Scope. We target dense transformer models whose layer order is static. We do not evaluate Mixture-of-Experts routing, multi-GPU or multi-tenant sharing of a CXL pool, KV-cache placement in CXL, or CXL switches. No CXL 3.x hardware was available: the CXL link is simulated with parameters anchored

in published measurements, and the hardware validation (Section 5.2) uses pinned host memory on a PCIe-attached GPU as a stand-in for the pool.

2. Background and related work

Reaching CXL memory from a GPU. CXL expanders have been studied mainly as a CPU-side capacity tier: Pond [3] and TPP [4] place or migrate pages between local DRAM and CXL memory, and DirectCXL [8] and the characterisations of Melody [9] and Sun et al. [10] report latencies of roughly 140–410 ns and device-dependent bandwidth. Work on GPUs is more recent and confirms the host-mediated nature of the path. TraCT [5] reports that a plain `cudaMemcpy` from a CXL-mapped region goes through a bounce buffer in host DRAM and must be avoided by page-locking the region so that the DMA engine reads the device directly; it measures 10.1 GB/s and 640 ns for a second-generation expander. ITME [6] prefetches model weights layer by layer from a CXL-hybrid memory into GPU memory with a DMA pipeline and reports about 18 GB/s for the prototype. DAK [11] instead lets the GPU read remote memory directly through the Tensor Memory Accelerator on NVLink-C2C and PCIe systems, and cache-coherent GPU–CPU links such as GH200 expose coherent memory to the GPU [12]; these are different access paths from the one we model. Compression raises the effective bandwidth of a CXL link [13] and is orthogonal to the policy studied here. Full-system CXL simulators with silicon validation, such as CXL-DMSim [14], model the device and host side in far more detail than we do but, to our knowledge, do not model GPU kernels.

Weight offloading for LLM inference. FlexGen [1] formulates the placement of weights, activations and KV cache over GPU, CPU and disk as a linear program and overlaps I/O with compute for throughput-oriented generation; ZeRO-Inference/Infinity [2] stream partitioned weights from CPU and NVMe. PowerInfer [7] keeps frequently activated (“hot”) neurons on the GPU and computes the rest on the CPU, exploiting activation sparsity rather than streaming weights. Serving systems batch iteration by iteration [15, 16], and chunked prefill [17] mixes prompt chunks with decode tokens, which is what makes the arithmetic intensity of an iteration a tunable quantity. KV-cache offloading [18, 19] and prefill/decode disaggregation [20] address different data and are complementary.

Table 1: Closest prior systems and CAMP along the dimensions relevant to this paper (as described by their authors).

System	Tier / path	Resident-set rule	Transfer scheduling	Evaluation
FlexGen [1]	GPU/CPU/disk, explicit copies	LP over weight/KV/activation fractions	overlapped I/O pipeline	real GPUs
PowerInfer [7]	GPU + CPU compute	hot neurons by profiled activation frequency	none (cold neurons run on CPU)	real consumer GPU
ITME [6]	CXL-hybrid memory $\rightarrow$ host $\rightarrow$ GPU	none described (layer-wise prefetch)	layer-wise DMA prefetch	hardware prototype
DAK [11]	GPU direct access (TMA) to remote memory	—	bypasses host prefetch	real NVLink-C2C / PCIe systems
CAMP (this work)	host-mediated CXL $\rightarrow$ GPU, explicit path model	SCP: planner + dry-run verification	work-conserving, next-use admission, online cost model	event-driven simulation; pipeline logic validated on a real GPU

Prefetching and replacement under a known access order. Prefetch and tiering policies for general workloads infer behaviour from pressure or access samples [21, 22]. When the future order is known, Belady’s optimal replacement [23] gives the offline reference; graph-aware memory planners exploit it for activation memory in training [24, 25]. For repeated cyclic scans of a model, recency-based replacement fails by construction, which is the behaviour that pinning is meant to break.

Positioning. Table 1 summarises the closest systems. The closest in spirit are FlexGen and ITME, which also overlap weight transfers with compute, and PowerInfer, which also partitions by frequency. CAMP differs in operating at layer granularity over a host-mediated CXL path with an explicit transfer model, in choosing the resident set by a pipeline-aware objective and a verification step instead of a frequency or prefix rule, and in deriving the compute estimate it needs at runtime. We do not claim that frequency-based hot/cold partitioning is inferior in general: in link-bound decode it is within 3% of SCP (Section 6.3).

3. System model

3.1. Host-mediated CXL-to-GPU path

Figure 1 shows the path that the simulator models. The CXL Type-3 expander is exposed to the operating system as a CPU-less NUMA node. The CAMP runtime, a host thread, page-locks the CXL-backed region holding the weights and issues one `cudaMemcpyAsync` per layer unit on a dedicated copy stream; the GPU copy engine then reads the data through the root complex, which forwards the reads to the expander using CXL.mem, and writes it into the HBM slot of the target layer. Kernels of the consuming layer wait on a CUDA event recorded after the copy. The initiator is therefore the host, the data crosses two links in series (CXL, then PCIe), and the sustained rate is bounded by the slower of the two:

$$BW_{\text{eff}} = \eta \min(BW_{\text{CXL}}, BW_{\text{PCIe}}), \quad t_{\text{copy}}(s) = t_0 + \left\lceil \frac{s}{c} \right\rceil t_c + t_{\text{sync}} + \frac{s}{BW_{\text{eff}}}, \quad (1)$$

where s is the transfer size, c the DMA chunk size, t_0 the per-copy API, driver and first-byte cost, t_c the per-chunk descriptor cost and t_{sync} the event synchronisation cost. If the region is *not* page-locked, the driver stages data through a bounce buffer in host DRAM; the CPU copy and the DMA are then serial and $BW_{\text{eff}} = (1/BW + 1/BW_{\text{stage}})^{-1}$ (“staged” mode). On a Tesla T4 we measured 12.4 GB/s for pinned and 4.6 GB/s for pageable copies, a ratio of 0.37 (Section 5.2), consistent with the bounce-buffer penalty reported for CXL regions [5].

The simulator additionally includes (i) a single non-preemptive copy engine that serves demand fetches before queued prefetches, (ii) log-normal noise on every copy and a slow autoregressive drift of the link bandwidth that models interference from other traffic, and (iii) a fixed host sampling gap between decode iterations during which the link keeps working. It does *not* model CXL switches or pooling across hosts, interleaving of several expanders (this is folded into BW_{eff} , which we sweep from 8 to 64 GB/s), GPU-initiated direct access to remote memory, cache-coherence traffic, or contention with host CPU memory traffic. Table 2 lists the parameters and their basis.

3.2. Layer units and iteration traces

A model is decomposed into *units*, the granularity at which weights are fetched and pinned: one attention block (QKV and output projections and norms), one feed-forward block, the embedding table and the output head

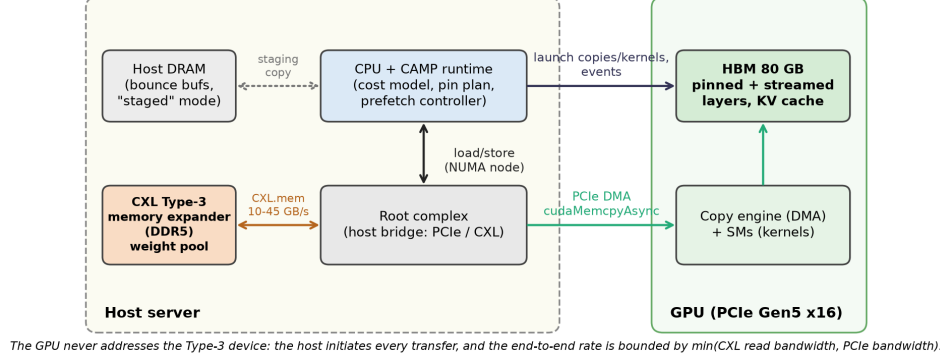


Figure 1: Host-mediated path modelled in this paper. The GPU does not address the CXL Type-3 expander; the CAMP runtime on the host issues DMA copies from page-locked CXL-backed memory into HBM, and synchronises kernels with events. “Staged” mode adds a bounce-buffer copy through host DRAM.

(tied models share one unit). Table 3 lists the configurations. An *iteration* is one pass over the units: a prefill pass over B prompts of S tokens, a decode step for B sequences with context length C , or a mixed iteration in which a prefill chunk piggybacks on decode tokens. The embedding access is a gather of T rows rather than a full-table fetch. The KV cache and workspace stay in HBM and reduce the capacity C_w available for weights: for Llama-3-70B with $B=16$ sequences of up to 4k tokens, the KV cache takes 21.5 GB, leaving $C_w = 54.5$ GB (39% of the weights).

3.3. Compute model and regimes

The execution time of a unit on the simulated GPU follows a roofline model [26] per sub-operation:

$$t = \max\left(\frac{F_g}{P\eta_g}, \frac{B_g}{W\eta_m}\right) + \max\left(\frac{F_a}{P\eta_a}, \frac{B_a}{W\eta_m}\right) + n_k t_{\text{launch}}, \quad (2)$$

where F_g, B_g are the FLOPs and HBM bytes of the projections, F_a, B_a those of the attention core (including the KV cache read), P the peak FLOP rate, W the HBM bandwidth, η the efficiencies and n_k the number of kernels. This is the *ground truth* of the simulated device; CAMP sees only the features $(F_g, B_g, F_a, B_a, n_k)$ and must estimate t (Section 4.1).

Table 2: Simulated platform. “Measured” entries are anchored on published device measurements or on our own T4 micro-benchmarks (Section 5.2); all values are exposed as parameters of the simulator.

Component	Value	Basis
GPU	H100-class: 989 TFLOP/s FP16 dense, 3.35 TB/s HBM, 80 GB	vendor data sheet; true efficiencies (GEMM 0.62, memory 0.86, attention 0.42) are hidden from CAMP
Host-to-GPU path	pinned DMA via root complex (“direct”); optional bounce-buffer path (“staged”)	[5]; staged/pinned ratio 0.37 measured on T4
Effective CXL→GPU bandwidth	18 GB/s (default, “measured”); 45 GB/s (“expected”, x16 Gen5); sweep 8–64 GB/s	≈ 10 GB/s device-level [5], ≈ 18 GB/s DMA to GPU [6]; PCIe Gen5 pinned copies are the upper bound
Per-copy fixed cost	12 μ s API/driver/first-byte + 6 μ s event sync + 3 μ s per 32 MB chunk	T4: 10–19 μ s fit of copy time vs. size; CXL.mem load latency ≈ 0.3 – 0.6 μ s [5, 9]
Staging hop (staged mode)	8 GB/s CPU copy, serial with DMA	T4 pageable-copy measurement (4.6 vs. 12.4 GB/s)
Noise	per-copy log-normal $\sigma = 0.06$; slow AR(1) bandwidth drift $\sigma = 0.05$ ($\tau \approx 2.5$ s); per-layer compute $\sigma = 0.03$	varied up to $8\times$ in Section 6.7
Host sampling gap	150 μ s between decode iterations (link keeps working)	typical scheduler/sampler cost

Streaming a unit of p parameters moves bp bytes ($b = 2$ for FP16) and enables $2pT$ FLOPs, so compute hides the copy when

$$T \geq T^* = (1 - \varphi) \frac{P\eta_g}{BW_{\text{eff}}} \cdot \frac{b}{2}, \quad (3)$$

with φ the fraction of weight bytes kept resident. For $P\eta_g = 613$ TFLOP/s, $\varphi = 0.4$ and $BW_{\text{eff}} = 18$ or 45 GB/s, $T^* \approx 20\text{k}$ or 8k tokens per iteration. Decode with $B \leq 512$ is therefore always link-bound, and the only lever is the number of bytes that cross the link per iteration. Section 6.1 confirms this threshold in the simulator.

3.4. A lower bound

An iteration cannot finish before its compute-only time $\sum_u t_u$, nor before the minimum number of streamed bytes has crossed the link. If any unit is

Table 3: Model configurations used in the evaluation (public model-card hyper-parameters; FP16 weights). Layer units are one attention block, one feed-forward block, the embedding table and the output head.

Model	Params (B)	Layers	d	Heads/KV	d_{ff}	Vocab	Embed.	Weights (GB)
llama2-7b	6.74	32	4096	32/32	11008	32k	untied	13.5
llama2-13b	13.02	40	5120	40/40	13824	32k	untied	26.0
llama3-8b	8.03	32	4096	32/8	14336	128k	untied	16.1
llama3-70b	70.55	80	8192	64/8	28672	128k	untied	141.1
llama3-405b	405.85	126	16384	128/8	53248	128k	untied	811.7
gemma2-9b	9.24	42	3584	16/8	14336	256k	tied	18.5

streamed, one slot as large as the largest streamed unit must stay free, so with W_{tot} total weight bytes and cache capacity C_w at least $W_{\text{tot}} - (C_w - s)$ bytes are streamed, where s is the size of the largest streamed unit (all larger units must then be pinned and fit in $C_w - s$). We take the smallest feasible s :

$$T_{\text{LB}} = \max \left(\sum_u t_u, \frac{\max\{0, W_{\text{tot}} - C_w + s_{\min}\}}{BW_{\text{eff}}} \right). \quad (4)$$

The bound ignores per-copy overheads and pipeline fill, and is exact in the link-bound regime for uniform units. It is implementation-independent: no policy can beat it, and it gives each result an absolute reference.

4. The CAMP policy

CAMP has three parts. All rely on the static unit order of the model (the “graph”), none on offline profiling of the model or workload.

4.1. Runtime cost model

The prefetcher and the pinning stage need an estimate $\hat{t}_u$ of each unit’s compute time for the *current* batch descriptor (B, S, C) . CAMP computes it from tensor shapes with the roofline of Eq. (2), using generic constants for the device class (efficiencies $\hat{\eta}_g = 0.50$, $\hat{\eta}_m = 0.80$, $\hat{\eta}_a = 0.35$) and the peak rates that the driver reports. These constants differ from the device’s true efficiencies, so the estimate is calibrated online: for each unit kind κ (attention, feed-forward, embedding, head) the runtime keeps a multiplicative correction updated by an exponential moving average of the ratio between observed and nominal duration,

$$\hat{c}_\kappa \leftarrow (1 - \alpha) \hat{c}_\kappa + \alpha \frac{t_u^{\text{obs}}}{t_u^{\text{nom}}}, \quad \hat{t}_u = \hat{c}_\kappa t_u^{\text{nom}}, \quad \alpha = 0.3. \quad (5)$$

The first iteration therefore runs with the generic estimate, and every later iteration, or any re-planning after the first probe iteration, uses the calibrated one. The current layer’s own duration is *not* needed before it completes: the estimate of unit u is available from its features before it runs, and the observation after it runs only refines future estimates. We quantify the effect of estimator error in Section 6.5.

4.2. Work-conserving prefetching with next-use admission

Because the unit order is static, the next use of every unit is known. Algorithm 1 issues prefetches whenever the link or a layer completes. A fetch of the unit needed at position j is admitted if free capacity exists or if some resident, unpinned unit has its next use *after* j (Belady-style admission [23]); otherwise the scan stops. The demand fetch of the layer about to run is always issued first, and prefetches cannot take capacity while a demand fetch is waiting for room. This controller is work-conserving: it never leaves the link idle if capacity allows another useful fetch. An earlier formulation limited the lookahead to the units whose cumulative transfer time fits in the estimated remaining compute of the current layer; in all our workloads that gate never binds, because completions re-trigger the controller, and results are identical with and without it (Section 6.5). The estimate $\hat{t}_u$ is thus not on the critical path of prefetching; it matters for pinning.

4.3. Stall-cost-aware pinning

Problem statement. Let U be the set of units with sizes s_u , C_w the weight-cache capacity, $\mathcal{P} \subseteq U$ the pinned set and $T_{\text{plan}}(\mathcal{P})$ the predicted steady-state iteration time of the pipeline planner defined below. SCP solves

$$\min_{\mathcal{P} \subseteq U} T_{\text{plan}}(\mathcal{P}) \quad \text{s.t.} \quad \sum_{u \in \mathcal{P}} s_u + \max_{v \in U \setminus \mathcal{P}} s_v \leq C_w, \quad (6)$$

that is, the pinned units plus one streaming slot large enough for the largest streamed unit must fit. This is *not* a 0–1 knapsack: T_{plan} is not additive over units and depends on where the pinned units sit in the sequence. The additive proxy

$$\max_{\mathcal{P}} \sum_{u \in \mathcal{P}} F(u) t_{\text{copy}}(s_u) \quad \text{s.t.} \quad \sum_{u \in \mathcal{P}} s_u \leq C_w - s^*, \quad (7)$$

with $F(u)$ the number of accesses of u per iteration, is a 0–1 knapsack whose benefit-per-byte is $F(u) t_{\text{copy}}(s_u) / s_u \approx F(u) / BW_{\text{eff}}$. For uniform sizes it ranks

Algorithm 1 Work-conserving prefetch controller (invoked at layer start, layer begin and every copy completion)

```

1: Input: access order  $Q$ , current position  $i$ , cache with pinned set  $\mathcal{P}$ 
2:  $j \leftarrow i + 1$ 
3: while  $j < |Q|$  and no demand fetch is blocked do
4:    $u \leftarrow Q[j]$ 
5:   if  $u$  is neither resident nor in flight then
6:     if free capacity  $\geq \text{size}(u)$  then
7:       issue prefetch( $u$ )
8:     else
9:        $v \leftarrow \arg \max_{w \text{ resident}, w \notin \mathcal{P}, w \neq Q[i]} \text{nextuse}(w)$ 
10:      if  $\text{nextuse}(v) > j$  then evict  $v$ ; issue prefetch( $u$ ) else break
11:      end if
12:    end if
13:  end if
14:   $j \leftarrow j + 1$ 
15: end while

```

units by $F(u)$ alone, which is the frequency rule of CAMP-v1 and of hot/cold partitioning. We solve Eq. (7) exactly by dynamic programming and use it as a baseline and as one SCP candidate; it is a proxy, and Section 6.3 shows when it is a poor one.

Pipeline planner. $T_{\text{plan}}(\mathcal{P})$ is evaluated with a deterministic recurrence over one iteration repeated three times. Let $\hat{c}_j$ be the estimated compute time of access j , L the time at which the link becomes free and $R = C_w - \sum_{\mathcal{P}} s_u$ the streaming buffer. A streamed access j is fetched as early as the link is free and the buffer has room, i.e. once earlier streamed units have finished computing and released enough bytes; it then computes when both the previous layer and its own fetch have finished:

$$f_j = \max(L, \rho_j) + t_{\text{copy}}(s_j), \quad L \leftarrow f_j, \quad (8)$$

$$b_j = \max(e_{j-1}, f_j), \quad e_j = b_j + \hat{c}_j, \quad (9)$$

where ρ_j is the earliest time at which the buffer holds at most $R - s_j$ bytes of streamed units, and pinned accesses skip the fetch. The planner costs $O(|Q|)$ per evaluation.

Algorithm 2 Stall-cost-aware pinning (SCP)

```

1: Input: unit sequence with sizes, cost model  $\hat{t}$ , link model  $t_{\text{copy}}$ , capacity  $C_w$ 
2:  $\mathcal{S} \leftarrow \{\emptyset\} \cup \{\text{STRIDE}(g) : g \in \{.3, .45, .6, .75, .9\}\} \cup \{\text{PREFIX}, \text{FREQ}, \text{KNAPSACKDP}, \text{GREEDY}\}$   $\triangleright$  feasible sets only
3:  $\mathcal{P}^* \leftarrow \arg \min_{\mathcal{P} \in \mathcal{S}} T_{\text{plan}}(\mathcal{P})$ 
4:  $\mathcal{P}^* \leftarrow \text{LOCALSEARCH}(\mathcal{P}^*)$   $\triangleright$  add / remove / swap within  $\pm 3$  positions
5:  $\mathcal{P}^* \leftarrow \text{ANNEAL}(\mathcal{P}^*, 1200 \text{ evaluations})$ 
6:  $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathcal{P}^*\}$ 
7: for  $\mathcal{P} \in \mathcal{S}$  do
8:    $\tau(\mathcal{P}) \leftarrow$  replay 3 iterations in the event engine with estimated layer times
9: end for
10: return  $\arg \min_{\mathcal{P} \in \mathcal{S}} \tau(\mathcal{P})$ 

```

Search and verification. Algorithm 2 shortlists structured candidates (no pinning, prefix, frequency-ranked, knapsack-DP, evenly interleaved “stride” sets at five pinned fractions, and the lazy marginal-gain greedy set), scores them with T_{plan} , refines the best by add/remove/local-swap search and a short simulated-annealing polish (about 1 200 planner evaluations), and finally replays the shortlisted sets in the event engine for three iterations *using the cost model’s estimated layer times*, never the device’s true times, and keeps the fastest replay. The replay removes planner approximations, such as ignoring buffer reuse across iterations, from the final decision. Planning takes 0.2–0.7 s for 160-unit models on one core and is performed once per operating point (or after calibration), not per token.

Why placement matters. Consider N equal units with compute c and fetch time $f > c$, a streaming buffer holding $m \geq 2$ units, and P pinned units. If the pinned units form a contiguous prefix, the link can only run $m - 1$ units ahead while they compute, and idles for the rest:

$$T_{\text{prefix}} \approx (N - P)f + \max\{0, Pc - (m - 1)f\}, \quad T_{\text{interleaved}} \approx \max\{(N - P)f, Nc\}. \quad (10)$$

The penalty vanishes when $c \ll f$ (decode), and is largest when compute and transfer are comparable. Frequency-ranked and prefix pinning, which break ties by sequence position, generate exactly the contiguous case; Section 6.3 measures the link utilisation they lose.

Transient layers and eviction. Units that are not pinned are evicted by next-use order (furthest next use first), which is optimal for the known order [23]; the admission rule of Algorithm 1 prevents a prefetch from displacing a nearer use. On a cyclic scan with equal frequencies, LRU, FIFO and LFU all evict the layer that will be needed soonest and retain nothing (Section 6.4).

5. Evaluation methodology

5.1. Simulator

The simulator (`camp_sim`, released with this paper) is an event-driven model of one GPU, one copy engine and an HBM weight cache. It contains no policy: prefetch, victim selection and pinning are plug-ins. The compute stream executes accesses strictly in order; each access first issues its demand fetch, then lets the policy issue lookahead, then waits for the weights. Copy completions and layer completions are events, and policies are re-invoked at both. Noise is generated per seed; the bandwidth drift is indexed by simulated time, so different policies run with the same seed see the same environment (common random numbers), and policy comparisons use paired seeds. All experiments use ten seeds unless stated; we report means and 95% confidence half-widths (t -distribution). Layer weights are never materialised: the simulator tracks bytes, not tensors.

Three things are checked independently of the experiments. (i) `tests/test_engine.py` compares the deterministic engine with closed-form pipeline times (demand paging $N(f + c)$; prefetch with a ring buffer $N \max(f, c)$; pinned layers $(N - P)f$; compute-bound Nc), which agree to within 2%. (ii) The analytic planner agrees with the engine to a mean absolute error of 0.18% (maximum 3.3%) over 80 random model/workload/cache/link configurations (Section 6.7). (iii) The pipeline logic is compared with a real GPU (Section 5.2).

5.2. Hardware validation of the pipeline logic

The simulator’s novel parts are the copy path and the overlap of transfers with compute. We validate them on a real GPU with CUDA streams and events (`validation/hw_validate.py`, run on a Tesla T4, PCIe Gen3 x16, CUDA 13). Twenty-four weight tensors of 64 MiB live in page-locked host memory, standing in for the CXL pool; layer u computes a GEMM of T tokens against its tensor. The loop supports a configurable prefetch depth (ring of device slots, events for slot reuse and copy completion) and a number of tensors pinned in HBM. We measure, *in the same loop*, (a) a compute-only

Table 4: Simulator versus a real GPU (Tesla T4, PCIe Gen3 x16, 24 weight tensors of 64 MiB streamed from pinned host memory). The simulator is composed from separately measured compute-only and copy parameters and asked to predict the measured composite iteration time; no parameter is fitted to the composites.

Regime	Configs	MAPE (%)	Mean signed (%)	Max err (%)
link-bound (compute $< 0.3\times$ copy)	10	4.0	+4.0	24.1
balanced ($0.3\text{--}1.2\times$)	10	8.9	-2.8	15.5
compute-bound ($> 1.2\times$)	5	5.6	-5.6	6.3
demand paging	5	14.6	+14.3	25.5
all configurations	30	7.6	+1.8	25.5

iteration (all tensors resident) for each T , (b) a copy-only iteration, and (c) 30 composite configurations (five T values $\times$ six combinations of depth and pinned count), three repetitions each. The simulator is then composed from the separately measured compute time per unit and the fitted copy cost (12.4 GB/s, 10.6 μ s) and asked to predict the composites. No parameter is fitted to the composites.

Figure 2 and Table 4 give the result. Across the 30 configurations the mean absolute error is 7.6% (median 5.6%). In the link-bound regime, where the pipeline logic dominates, the error is 4.0% on average and the 9 configurations with prefetch depth ≥ 1 , $T \leq 1024$ and no pinned tensors are within 1.3%. The largest errors (up to 25%) occur where compute time is a large share and the simulator’s constant compute time per unit is wrong: on this power-capped 70 W GPU, the same GEMM took about half as long when interleaved with copies at low duty cycle (demand paging) as in the back-to-back compute-only loop, and about 6% longer under sustained load, a clock and power effect that the simulator does not model. The result supports the copy path, queueing and overlap logic; it does not validate the absolute compute times of an H100 or any CXL device, which we take from the roofline model and published measurements.

5.3. Workloads, baselines and metrics

Workloads use the models of Table 3. For decode we use $B = 16$ sequences with 2k-token contexts (70B: KV capacity reserved for 4k), for prefill $B \in \{1, 4\}$ prompts of 2k or 4k tokens, and for mixed iterations 64 decode tokens plus a 512-token prefill chunk; the 13B/9B models are given 40% of their weight bytes as cache (as on a GPU shared with other tenants or a large

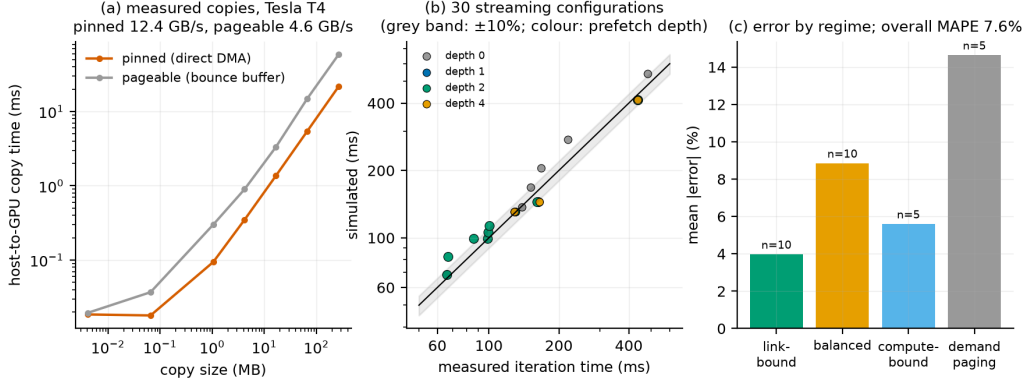


Figure 2: Validation on a Tesla T4. (a) Measured host-to-GPU copy time versus size for pinned and pageable memory (pageable reaches 37% of the pinned bandwidth, the bounce-buffer penalty). (b) Simulated versus measured iteration time for 30 streaming configurations. (c) Mean absolute error by regime; ratio of compute to copy time per unit defines the regime.

KV cache), the 70B model the HBM left after its KV cache. Non-uniform reuse is evaluated with Gemma-2-9B (tied embedding and head), speculative decoding (13B target, 1B draft, $k = 4$ draft steps per verification) and a 24+24-layer encoder-decoder processing 64 output tokens per request.

Baselines (all given the same cache capacity and link): **Demand+LRU** (no prefetch); **Static- k** (fixed lookahead, best $k \in \{1, 2, 4, 8\}$ chosen per configuration, LRU); **Reactive** (lookahead grows on observed stalls, decays when stall-free; a pressure-driven, TMO-like controller [21]); **Aggressive-10** (fixed deep lookahead, Limoncello-like [22]); **FlexGen-style** (resident layer prefix, double-buffered streaming [1]); **Hot/Cold** (resident set chosen by access frequency, ties by position, streaming with lookahead 2; PowerInfer-style partitioning [7] adapted to streaming); **CAMP-v1** (the earlier formulation: lookahead gated by the compute estimate, frequency pinning); and **CAMP-v2** (this paper). These are reimplementations of the *mechanisms* in the common simulator, not ports of the original systems. CAMP-v2 uses the cost model calibrated after one probe iteration and SCP with dry-run verification; variants with the generic (uncalibrated) and the exact device model are reported where relevant.

The metric is the steady-state iteration latency (the first iteration is a warm-up and excluded), compared with the lower bound of Eq. (4).

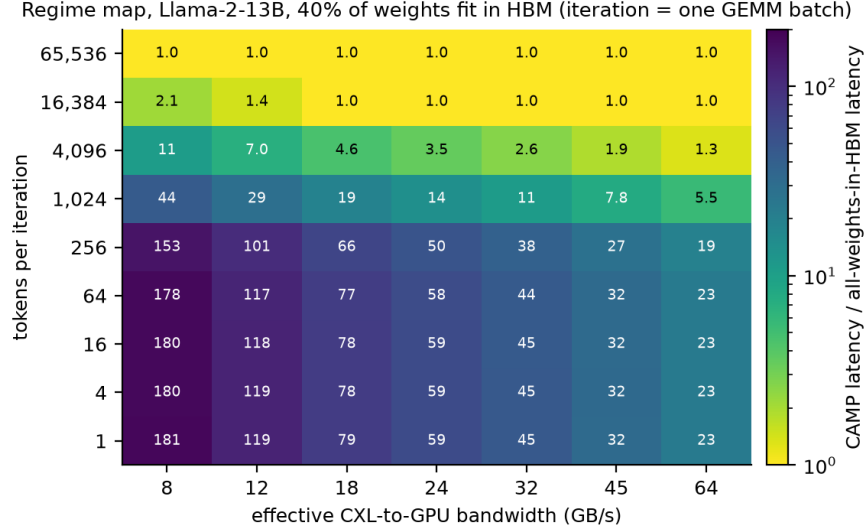


Figure 3: Regime map: CAMP iteration latency relative to all-weights-in-HBM, Llama-2-13B with 40% of the weights cached, as a function of tokens per iteration and effective CXL-to-GPU bandwidth (each cell: four seeds).

6. Results

6.1. Regime map: when does streaming from CXL pay off?

Figure 3 gives the latency of CAMP-managed streaming relative to a GPU with all weights resident, for Llama-2-13B with 40% of the weights cached, as a function of tokens per iteration and of BW_{eff} . At 18 GB/s the slowdown is $79\times$ for one token, $77\times$ at 64 tokens, $66\times$ at 256, $19\times$ at 1024, $4.6\times$ at 4096, $1.14\times$ at 16384 and $1.01\times$ at 65536. At 45 GB/s the same rows read $32\times$, $32\times$, $27.5\times$, $7.8\times$, $1.9\times$, $1.01\times$ and $1.00\times$. The transition follows Eq. (3), $T^* \approx 20\text{k}$ tokens at 18 GB/s and 8000 at 45 GB/s. Two consequences follow. Prefill of long prompts and very large batches can run from CXL-resident weights at close to resident speed, so capacity can be traded for little latency. Interactive decode cannot: with at most 256 tokens per iteration the slowdown is between $19\times$ (256 tokens, 64 GB/s) and $181\times$ (one token, 8 GB/s), and the only lever is to move fewer bytes. Neither prefetching nor cleverer replacement changes this; a larger cache, a faster link or compression do [13].

6.2. Main comparison

Table 5 and Figure 4 compare the policies in 16 configurations (three models, decode, mixed and prefill iterations, two link bandwidths). Three observations.

CAMP-v2 is close to the lower bound everywhere. Its latency is $0.996\text{--}1.058\times$ the bound of Eq. (4) in all 16 configurations (values just below one are noise around a deterministic bound), with the largest gaps in the two balanced 18 GB/s prefill configurations (1.054 and 1.058).

Against published-style baselines the gain depends on the regime. In link-bound decode and short prefill, where the number of resident bytes decides everything, CAMP-v2 is $1.04\text{--}1.11\times$ faster than the FlexGen-style and Hot/Cold baselines and $1.6\text{--}1.8\times$ faster than demand paging or any LRU-based prefetcher, which retain nothing on a cyclic scan. In the balanced prefill configurations (13B $B=4$, $S=4\text{k}$ at 18 GB/s; 70B prefill at 18 and 45 GB/s) it is $1.26\text{--}1.47\times$ faster than FlexGen-style and Hot/Cold, $1.23\text{--}1.40\times$ faster than CAMP-v1, and $2.4\text{--}2.9\times$ faster than demand paging. In compute-bound configurations (13B and 70B, 4k-token prefill at 45 GB/s) the best static baselines already reach the bound and CAMP-v2 matches them (within 1%); in the 13B case the Hot/Cold baseline is 29% slower because its pinned set and fixed lookahead leave too small a streaming buffer (Fig. 4).

The tied-embedding model benefits from a different pin set. For Gemma-2-9B, CAMP-v2 is $1.29\times$ faster than FlexGen-style and Hot/Cold. The reason is mechanical: the baselines reserve two slots of the largest unit (the 1.8 GB embedding/head table) for streaming, which caps what they can pin at 3.7 GB of 7.4 GB; SCP pins the table itself and keeps 7.0 GB resident.

6.3. Pinning: what does the selection rule buy?

Figure 5 compares pin-selection rules with the prefetcher fixed (CAMP’s controller, next-use eviction): no pinning, prefix, frequency-ranked (the pinning rule of CAMP-v1), random, interleaved stride, the exact knapsack of Eq. (7), and SCP.

Link-bound iterations are insensitive to the rule. In decode and short prefill every rule that pins about 9–10 GB lands within 3% of the others (13B decode at 18 GB/s: frequency 898 ms, random 899 ms, stride 909 ms, knapsack 891 ms, SCP 873 ms); what matters is the number of resident bytes. SCP’s edge of about 3% there comes from packing: it keeps 9.9 GB resident where the conservative reserve of the other rules allows 9.3 GB.

Table 5: Iteration latency (ms, mean of 10 seeds) of CAMP-v2 and the baselines. “Cache” is the share of weight bytes that fit in the HBM weight cache. Speed-up is relative to the best baseline in the row; “/LB” is CAMP-v2 latency divided by the lower bound of Section 3.4. Static- k uses the best $k \in \{1, 2, 4, 8\}$ per configuration. 95% confidence half-widths are $< 3\%$ of the mean for every entry (Fig. 4).

Model	Workload	Cache	GB/s	Demand	Static- k	FlexGen	Hot/Cold	CAMP-v1	CAMP-v2	LB	Speed-up	/LB
Llama-2-13B	dec. $B=16, 2k$	40%	18	1 425	1 403	915	905	898	873	873	$1.03\times$	1.00
	dec. $B=16, 2k$	40%	45	587	566	374	370	363	353	349	$1.03\times$	1.01
	mixed 64+512	40%	18	1 468	1 403	931	920	898	877	873	$1.02\times$	1.00
	mixed 64+512	40%	45	630	566	389	384	364	356	349	$1.02\times$	1.02
	pre. $B=1 S=2k$	40%	18	1 494	1 403	942	933	903	884	873	$1.02\times$	1.01
	pre. $B=1 S=2k$	40%	45	658	568	399	395	375	359	349	$1.05\times$	1.03
	pre. $B=4 S=4k$	40%	18	2 160	1 410	1 167	1 157	1 129	920	873	$1.23\times$	1.05
	pre. $B=4 S=4k$	40%	45	1 314	757	750	970	750	750	746	$1.00\times$	1.01
	dec. $B=16, 2k$	39%	18	7 895	7 837	5 124	5 092	5 074	4 880	4 772	$1.04\times$	1.02
	dec. $B=16, 2k$	39%	45	3 145	3 084	2 012	1 999	1 980	1 908	1 909	$1.04\times$	1.00
Llama-3-70B	pre. $B=4 S=2k$	52%	18	9 757	7 836	5 027	4 996	4 676	3 807	3 728	$1.23\times$	1.02
	pre. $B=4 S=2k$	52%	45	5 015	3 060	2 500	2 488	2 390	1 905	1 887	$1.25\times$	1.01
	pre. $B=4 S=4k$	50%	18	11 731	7 844	6 036	6 036	5 725	4 103	3 877	$1.40\times$	1.06
	pre. $B=4 S=4k$	50%	45	7 033	3 907	3 879	3 879	3 879	3 880	3 874	$1.00\times$	1.00
	dec. $B=32, 2k$	40%	18	1 022	1 012	816	816	813	631	633	$1.29\times$	1.00
	dec. $B=32, 2k$	40%	45	423	409	332	332	329	254	253	$1.29\times$	1.00
Gemma-2-9B	dec. $B=32, 2k$	40%	18	1 022	1 012	816	816	813	631	633	$1.29\times$	1.00
	dec. $B=32, 2k$	40%	45	423	409	332	332	329	254	253	$1.29\times$	1.00

Balanced iterations are very sensitive to placement. In the three 18 GB/s prefill configurations with compute comparable to transfer, frequency-ranked pinning is $1.23\text{--}1.40\times$ slower than SCP, and even random pinning is $13\text{--}25\%$ faster than frequency ranking. For 13B prefill ($B=4, S=4k$) the link utilisation is $79\text{--}80\%$ for prefix and frequency pins (which pin a contiguous run of early layers, so the link runs ahead by only a few units while they compute) against $99.5\text{--}100\%$ for random, stride and SCP sets; the iteration times are 1 167 versus 927–935 ms in the deterministic engine. This is the mechanism of Eq. (10). The exact knapsack (DP) packs more bytes and beats frequency ranking, but remains $6\text{--}31\%$ slower than SCP because its benefit ignores overlap. In the compute-bound 4k-token prefill configurations at 45 GB/s all rules are within 4% of each other and of no pinning; the 70B 2k-token prefill at 45 GB/s is balanced again, and frequency ranking is 25% slower than SCP there.

Optimality. On an 18-unit model, where all 2^{18} pin sets can be enumerated, SCP is on average 0.7% (maximum 4.2%) above the exhaustive optimum of the planner objective, versus 21.1% (maximum 32.4%) for frequency ranking (Table 6); replayed in the event engine the mean gap is 2.7% and the maximum 12.8%, which marks the planner’s approximation error that the dry-run verification reduces but does not remove. SCP is a heuristic with no optimality guarantee; the simple stride rule, which needs no planner, is within 1% of SCP in six of the twelve cases and within 4.5% in eleven of them (8.2% for 70B 2k-token prefill at 18 GB/s).

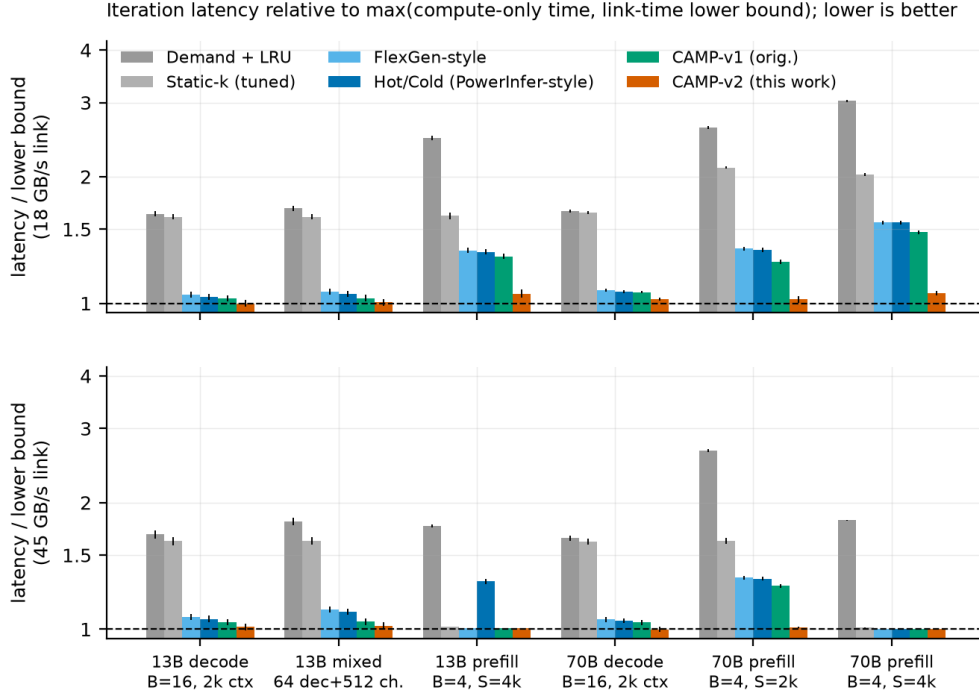


Figure 4: Iteration latency relative to $\max(\text{compute-only time}, \text{link lower bound})$ at 18 and 45 GB/s (mean of 10 seeds, 95% confidence bars). Dashed line: lower bound.

6.4. Non-uniform reuse and eviction

Eviction on a uniform loop. Table 7 shows eviction policies on 13B decode with no pinning. LRU, FIFO and LFU produce identical results and zero hits: on a cyclic scan with equal access counts, LFU has no frequency signal to exploit and degenerates to LRU. Random eviction achieves 7% hits, next-use eviction 32.5%, and the latter matters because it is the only policy that retains some layers at all. A tie between LFU and graph-aware eviction can nevertheless appear when a model has tied embeddings, the only layers with a frequency difference (Gemma-2-9B below); it reflects that single signal, not an equivalence of the two mechanisms.

Workloads with genuinely different reuse. Table 8 and Figure 6 give three cases where the number of accesses per iteration differs between layers. For Gemma-2-9B with a tied embedding, demand paging with LFU is 10% faster than LRU (the embedding is accessed twice per step), prefix, frequency and

Table 6: Optimality gap of the pin-set selection on an 18-unit model (8 layers, $d=2048$), where all 2^{18} pin sets can be enumerated. Planner gap: predicted iteration time of the selected set relative to the exhaustive optimum of the planner objective; simulated gap: the same sets replayed in the event engine.

Workload	Cache ratio	SCP gap (%)	Frequency gap (%)	SCP, simulated (%)
decode $B=16$	0.3	0.0	17.7	0.0
decode $B=16$	0.5	0.0	25.5	0.3
decode $B=16$	0.7	0.0	32.4	0.0
prefill $B=1$ $S=2k$	0.3	0.0	16.1	0.0
prefill $B=1$ $S=2k$	0.5	0.0	24.0	0.0
prefill $B=1$ $S=2k$	0.7	0.5	31.0	1.8
prefill $B=4$ $S=4k$	0.3	0.0	13.0	1.9
prefill $B=4$ $S=4k$	0.5	0.0	17.8	3.5
prefill $B=4$ $S=4k$	0.7	3.8	22.4	12.8
prefill $B=8$ $S=2k$	0.3	0.0	13.0	0.2
prefill $B=8$ $S=2k$	0.5	0.0	17.8	0.1
prefill $B=8$ $S=2k$	0.7	4.2	22.8	12.3

knapsack pinning give 0.81 s, and SCP 0.63 s thanks to pinning the large tied table. In speculative decoding the draft model’s layers are used $k = 4$ times per iteration; frequency pins already capture this (1.03 s versus 1.01 s for SCP), and LFU is 6% faster than LRU. For the encoder–decoder, encoder layers are used once per request and decoder layers 64 times: prefix pinning is useless (27.0 s), LRU and LFU are identical (27.8 s, since no layer is hit often enough for LFU to separate them), while frequency pins (10.2 s) and knapsack/SCP pins (9.1 s) are $2.7\text{--}3.0\times$ faster. Reuse awareness is therefore decisive when per-layer frequencies differ by large factors; it is not when they are uniform, and the cases in which CAMP’s pin rule differs from frequency ranking by more than a few percent are those with a very large unit (Gemma) or comparable compute and transfer (Section 6.3).

6.5. How much does the compute estimate matter?

Table 9 and Fig. 7 address how $\hat{t}_u$ is obtained and what happens when it is wrong. The generic roofline estimate has a mean absolute error of 23% for prefill and 9% for decode against the simulated device; one probe iteration reduces this to 0.5–2.7%. Prefetching is insensitive to the estimate: with the original compute-gated controller and with the work-conserving controller,

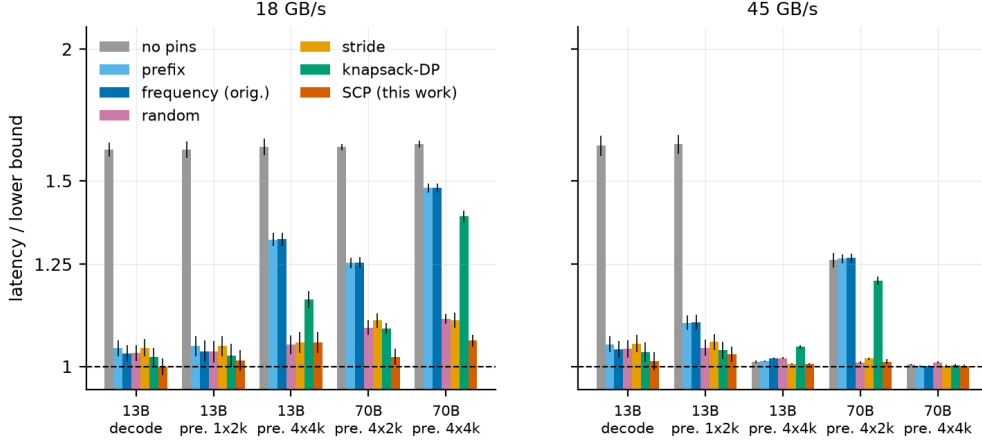


Figure 5: Pin-selection rules with the prefetcher and eviction fixed (latency relative to the lower bound; 10 seeds, 95% confidence bars). Placement matters when compute and transfer are comparable (13B and 70B prefill at 18 GB/s) and not in link-bound decode.

and with estimate biases from $0.25\times$ to $4\times$, the latencies are identical (919 ms, 3 788 ms and 873 ms for the three workloads). The pinning stage is sensitive in the balanced regime when the estimate is uncalibrated: for 13B prefill, SCP planned from an estimate with bias 0.5 or 0.25 gives 966 or 988 ms instead of 920 ms (+5% and +7%), and per-layer log-normal noise with $\sigma = 0.5$ adds up to 9% at bias 1; for 70B prefill a bias of 2 or 4 gives 4 712 or 4 671 ms instead of 3 788 ms, a 23–24% loss. After online calibration the spread over the whole bias range shrinks to 907–929 ms (13B) and 3 770–3 788 ms (70B), below 3% and still below 3% with $\sigma = 0.5$ noise. Decode is insensitive throughout (870–875 ms) because it is link-bound. In summary, CAMP needs the compute estimate only for pinning and only in the balanced regime, and an exponential moving average over one iteration of observed layer times is enough.

6.6. Batching, model size and link sensitivity

Batching. Figure 8 shows throughput. For decode with 512-token contexts (13B, 40% cached, 18 GB/s) CAMP reaches 1.1, 18.4, 73.5, 292 and 583 tokens/s at $B = 1, 16, 64, 256, 512$: throughput grows linearly with the batch because the streamed bytes per iteration are constant and compute is negligible, and the slowdown relative to resident weights shrinks from $79\times$ to $9.0\times$ at $B = 512$. Resident weights saturate at about 5 200 tokens/s (compute-

Table 7: Eviction policies on a uniform dense decode loop (Llama-2-13B, $B=16$, 2k context, 40% cached, 18 GB/s, no pinning). On a cyclic access pattern with equal frequencies, LRU, FIFO and LFU are indistinguishable (zero hits); only next-use eviction with the known layer order retains useful layers.

Eviction	demand paging		prefetch, depth 2	
	latency (ms)	hit (%)	latency (ms)	hit (%)
LRU	1 431	0.0	1 410	0.2
FIFO	1 431	0.0	1 410	0.2
LFU	1 431	0.0	1 410	0.2
Random	1 302	7.3	1 264	9.3
Next-use (graph-aware)	873	32.5	861	32.7

Table 8: Workloads with non-uniform reuse (latency in seconds; 18 GB/s; 35–40% of weights cached). The first two columns are demand paging with LRU/LFU eviction; the others use CAMP’s prefetcher with the stated pinning rule.

Workload	LRU	LFU	none	prefix	freq.	DP	SCP
Gemma-2-9B, tied embedding (per step)	1.03	0.93	1.01	0.81	0.81	0.81	0.63
speculative decoding, 13B+1B, $k=4$ (per step)	1.53	1.43	1.51	1.03	1.03	1.02	1.01
encoder–decoder, 64 tokens (per request)	27.80	27.80	27.18	27.02	10.21	9.15	9.14

and KV-bound) beyond $B \approx 256$. In the GEMM-batch view the throughput of CAMP saturates at the compute-bound limit (17 000 tokens/s) only at about 16 000 tokens per iteration. Batching amortises weight traffic, as in resident-weight engines, but the amortisation continues for two orders of magnitude longer in streaming mode because it starts from a link-bound state.

Model size. Figure 9 moves from 7B to 405B parameters with a fixed 80 GB HBM. The 7B model fits and every policy is equal. For 13B, the KV cache of 16 sequences leaves 86% of the weights resident, and CAMP (212 ms) is $1.6\times$ faster than the FlexGen-style baseline (337 ms) because it keeps all but one streaming slot resident. At 70B (39% cached) decode takes 4.9 s at 18 GB/s and 1.9 s at 45 GB/s, $1.6\times$ faster than demand paging; at 405B (5% cached) CAMP is at most 6% faster than demand paging (43.3 versus 45.5 s at 18 GB/s) because almost nothing can stay resident. The curves are not linear in model size because the cached fraction falls as the model grows; the knee between 13B and 70B marks the move from a mostly resident to a

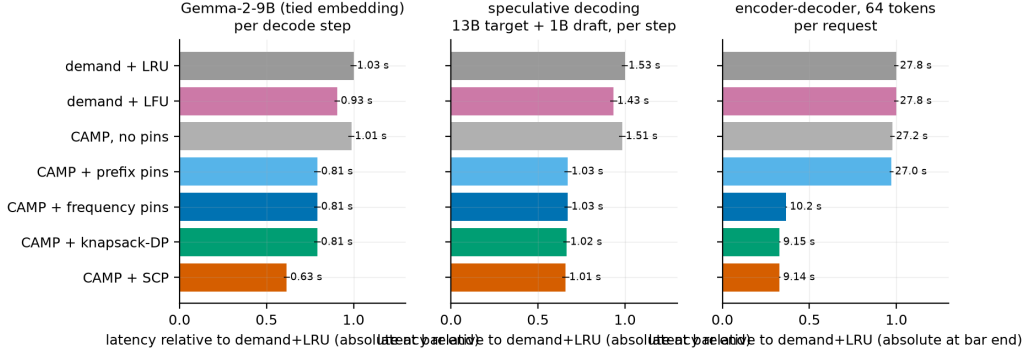


Figure 6: Workloads with non-uniform reuse (18 GB/s; 10 seeds). Bars: latency relative to demand+LRU; absolute value at the bar end.

Table 9: Runtime cost model. Mean absolute percentage error (MAPE) of the per-layer compute-time estimate before and after online calibration with one probe iteration, and the resulting iteration latency (ms) with SCP pins planned from the generic roofline estimate (“generic”), from the exact device model (“exact”), and without pinning.

Workload	MAPE generic (%)	MAPE calibrated (%)	no pins	generic
llama2-13b prefill B=4 S=4k	23.2	2.7	1 408	920
llama3-70b prefill B=4 S=2k	23.5	0.5	6 006	3 778
llama2-13b decode B=16	9.3	2.7	1 400	871

mostly streamed model.

Link, cache and path. Figure 10 sweeps BW_{eff} from 8 to 64 GB/s. In decode, CAMP follows the lower bound within 2.2% over the whole range, and Hot/Cold is 3.6–4.8% above CAMP. In balanced prefill (13B, $B=4$, $S=4k$) CAMP is within 3–5% of the bound from 8 to 18 GB/s and within 1.3% from 24 GB/s up; Hot/Cold is 7–29% slower than CAMP across the range, with the largest gaps near the compute-bound threshold. Sweeping the cache share from 10% to 100% at 18 GB/s keeps CAMP within 8.3% of the bound at every point (the two worst points, 1.083, are balanced prefill with a 50% cache and decode with an 80% cache), while Hot/Cold reaches 1.50 times the bound; the non-monotonic Hot/Cold curve comes from its conservative streaming-buffer reserve interacting with the unit sizes. The staged path is a large penalty: with bounce-buffer copies the effective bandwidth drops to 5.5 GB/s (from 18) and 6.8 GB/s (from 45), and CAMP’s decode latency

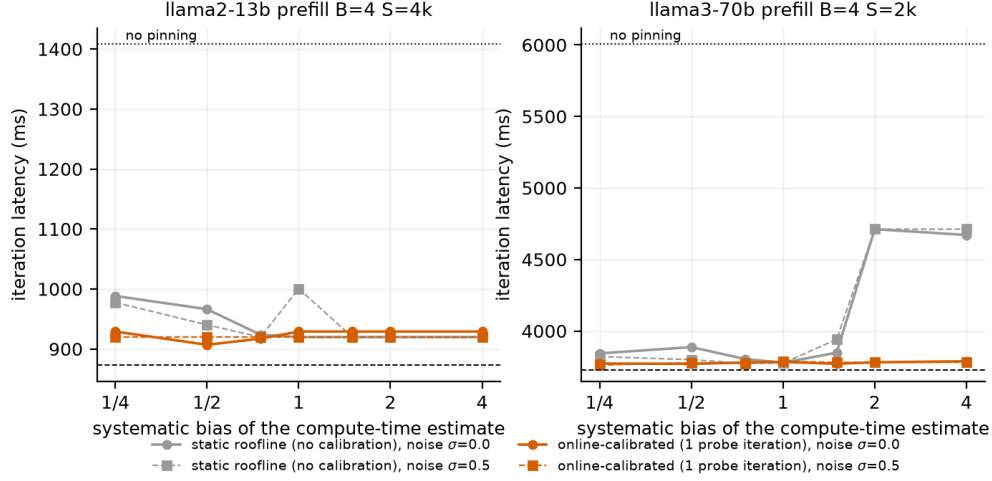


Figure 7: Iteration latency when SCP is planned from a compute estimate with systematic bias (x-axis) and per-layer noise, with and without online calibration. Dotted: no pinning; dashed: lower bound.

risers $3.2\times$ and $6.5\times$, so page-locking the CXL region is essential.

6.7. Robustness of the conclusions

Figure 11a scales jitter, drift and compute noise together from zero to eight times the default (log-normal copy jitter up to $\sigma = 0.5$, drift up to 0.4, compute noise up to 0.25). The ratio between CAMP-v2 and the Hot/Cold baseline stays within 0.96–0.97 in decode and 0.79–0.80 in balanced prefill; against demand paging it stays within 0.61–0.62 and 0.42–0.44. The ordering and the size of the gains are therefore not artefacts of a noise level, and the latency curves are not artificially smooth: the coefficient of variation of CAMP’s latency across seeds grows from 1.6–1.9% to 10–21% across the same sweep. Figure 11b shows the planner against the deterministic engine for 80 random configurations (five models, decode and prefill, cache shares of 15–70%, six link rates): mean absolute error 0.18%, median 0.01%, maximum 3.3%.

7. Discussion and limitations

What the results mean for CXL-backed inference. With an effective bandwidth of 10–20 GB/s per expander, streaming weights from CXL is practical for

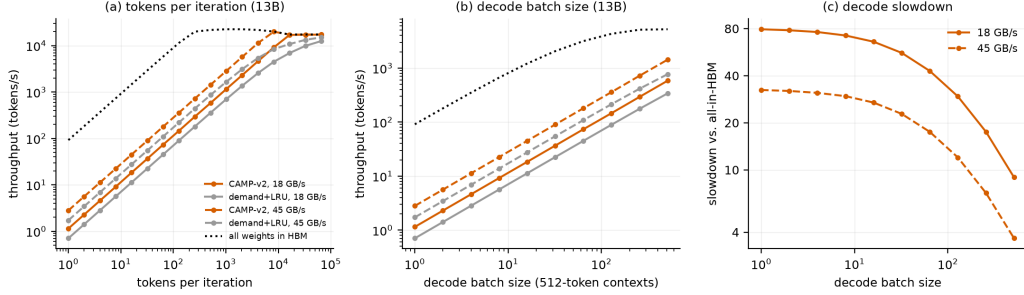


Figure 8: Throughput versus batch size, Llama-2-13B with 40% of the weights cached. (a) Tokens per iteration (GEMM batch). (b) Decode batch size with 512-token contexts. (c) Slowdown versus resident weights.

compute-dense iterations (long-prompt prefill, very large batches) and impractical for interactive decode, where the slowdown against resident weights is tens of times and prefetching cannot help because the iteration is link-bound. For decode, the achievable gains come from the number of resident bytes and the packing of the cache (Section 6.3), and from raising BW_{eff} (more expanders, page-locked DMA, compression). CAMP reaches the lower bound in link-bound configurations, so there is little room for further policy work there; the remaining room is in balanced iterations (up to 6% above the bound at 18 GB/s) and in the planner’s approximation error.

Limitations. (i) *No CXL hardware.* The CXL link is simulated; its bandwidth and fixed costs are taken from published measurements [5, 6] and swept over a wide range, and the pipeline logic is validated on a PCIe-attached T4 with pinned host DRAM in place of a CXL pool. We did not measure a real expander, a PCIe Gen5 GPU or an H100; the compute times of the H100-class device come from a roofline model with efficiencies that we did not measure. (ii) *Power and clock effects.* On the T4 the compute time of a layer depends on duty cycle by up to a factor of two, an effect absent from the simulator; it causes the largest validation errors (up to 25%) where compute and copy are comparable, which is exactly where SCP’s gains are, so the quantitative gains in the balanced regime should be read with this uncertainty. (iii) *Scope of the models.* We evaluate dense transformers with a static order. Mixture-of-Experts routing makes the access order data-dependent; hot/cold partitioning by observed frequency [7] is the natural baseline and an online variant of SCP is future work. (iv) *Single GPU, single tenant.* Multi-GPU

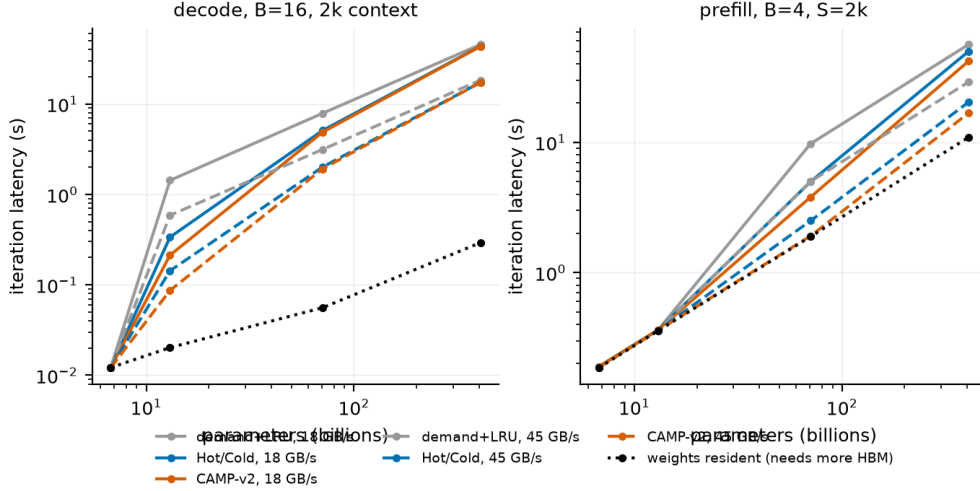


Figure 9: Iteration latency versus model size at fixed 80 GB HBM (B=16 decode with 2k context; B=4, S=2k prefill). The dotted line is the latency if all weights were resident.

sharing of one CXL pool, link contention between tenants and CXL switches are not modelled; . (v) *The KV cache stays in HBM*; offloading it to CXL [5, 18] competes for the same link. (vi) *SCP is a heuristic*. It is within 4.2% of the exhaustive optimum of its own objective on an 18-unit model, but there is no guarantee for larger models, and in the event engine the pin set it selects is up to 12.8% slower than the exhaustive optimum of the planner objective, an approximation error that the dry-run replay mitigates but does not remove. (vii) *Baselines are reimplementations* of mechanisms, not ports of FlexGen, PowerInfer, TMO or Limoncello, and the Hot/Cold baseline’s streaming buffer rule (two slots of the largest unit) is a choice that disadvantages it when one unit is very large. (viii) *Validation depth*. We do not use a cycle-accurate simulator; GPU kernels are black boxes at the level of detail we need, and CXL-DMSim-style device models [14] are complementary. The cost of re-planning pins when the batch descriptor changes materially (moving up to the pinned bytes once, about 0.5 s for 9 GB at 18 GB/s) is not charged in the experiments.

8. Conclusion

We studied LLM inference with weights streamed from CXL expanders through a host-mediated DMA path, showed with a closed-form threshold and

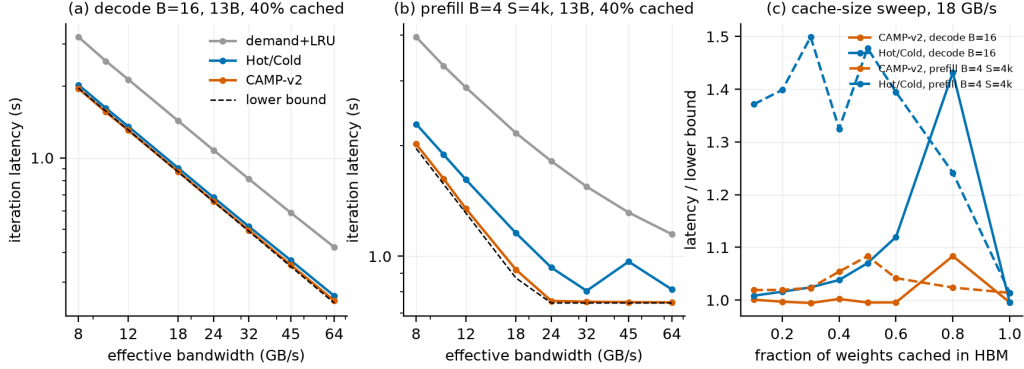


Figure 10: (a, b) Iteration latency versus effective bandwidth for decode and balanced prefill (13B, 40% cached). (c) Latency relative to the lower bound versus the cached fraction of the weights at 18 GB/s.

a regime map when this is nearly free (above about 16k tokens per iteration) and when it is tens of times slower (decode), and presented CAMP, a policy built from an online-calibrated cost model, a work-conserving prefetcher with next-use admission and stall-cost-aware pinning. The pipeline machinery of the simulator was validated on a real GPU (7.6% mean error, 4.0% when link-bound). In 16 configurations CAMP stays within 6% of an implementation-independent lower bound and is up to $1.47\times$ faster than the best static baseline. Its advantage over a frequency-based pinning rule is small in link-bound decode and large when compute and transfer are comparable, because contiguous pinned sets idle the link; LFU gives no protection on cyclic scans, and reuse-aware pinning is decisive when per-layer reuse genuinely differs, as in encoder-decoder inference. The main open items are measurement on real CXL hardware and PCIe Gen5 GPUs, online variants for Mixture-of-Experts, and multi-tenant sharing of the pool.

CRedit authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision.

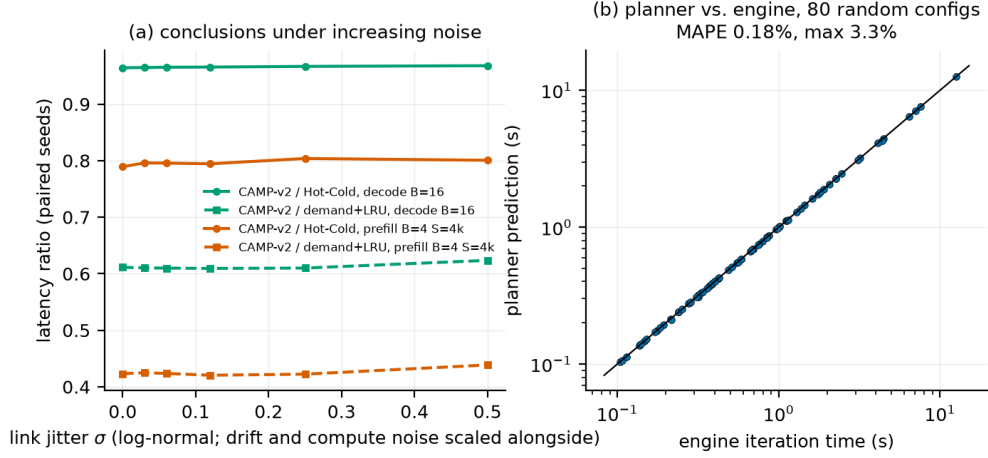


Figure 11: (a) Latency ratios of CAMP-v2 to baselines under increasing noise (paired seeds). (b) Pipeline planner versus deterministic event engine on 80 random configurations.

Data availability

The simulator, experiment scripts, raw result files, the hardware micro-benchmark and its measured output are available at <https://github.com/vinhqdang/Enabling-CXL-Based-Memory-Disaggregation-for-Large-Scale-AI-Systems>. Every figure and table in this paper is generated from the files in `results/` by `experiments/plots.py` and `experiments/make_tables.py`.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used AI-assisted tools to support software implementation, experiment scripting and drafting of text. After using these tools, the author reviewed, verified and edited all content as needed and takes full responsibility for the content of the publication.

References

- [1] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, C. Zhang, FlexGen: High-throughput generative inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML '23), 2023. URL <https://proceedings.mlr.press/v202/sheng23a.html>
- [2] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, Y. He, ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2021.
- [3] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, R. Bianchini, Pond: Cxl-based memory pooling systems for cloud platforms, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 574–587. doi:10.1145/3575693.3578835. URL <https://doi.org/10.1145/3575693.3578835>
- [4] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, P. Chauhan, Tpp: Transparent page placement for cxl-enabled tiered-memory, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 742–755. doi:10.1145/3582016.3582063. URL <https://doi.org/10.1145/3582016.3582063>
- [5] D. Yoon, Y. Min, H. Kim, S. H. Noh, J. Kim, TraCT: Disaggregated LLM serving with CXL shared memory KV cache at rack-scale (2025). arXiv:2512.18194.
- [6] H. Jang, Y. Min, S. Kim, T. Ahn, H. Kim, Y. Joo, H. Kim, J. Kim, ITME: Inference tiered memory expansion with disaggregated CXL-hybrid memories (2026). arXiv:2606.12556.

- [7] Y. Song, Z. Mi, H. Xie, H. Chen, PowerInfer: Fast large language model serving with a consumer-grade GPU, in: Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24), ACM, New York, NY, USA, 2024, pp. 590–606. doi:10.1145/3694715.3695964. URL <https://arxiv.org/abs/2312.12456>
- [8] D. Gouk, S. Lee, M. Kwon, M. Jung, Direct access, high-performance memory disaggregation with directcxl, in: 2022 USENIX Annual Technical Conference, USENIX ATC '22, USENIX Association, Berkeley, CA, USA, 2022, pp. 287–294. URL <https://www.usenix.org/conference/atc22/presentation/gouk>
- [9] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, H. Li, Systematic cxl memory characterization and performance analysis at scale, in: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '25, ACM, New York, NY, USA, 2025, pp. 404–418. doi:10.1145/3676641.3715987. URL <https://doi.org/10.1145/3676641.3715987>
- [10] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, N. S. Kim, Demystifying cxl memory with genuine cxl-ready systems and devices, in: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO '23, ACM, New York, NY, USA, 2023, pp. 105–121. doi:10.1145/3613424.3614256. URL <https://doi.org/10.1145/3613424.3614256>
- [11] S. Lin, Z. Guo, J. Lin, DAK: Direct-access-enabled GPU memory offloading with optimal efficiency for LLM inference (2026). [arXiv:2604.26074](https://arxiv.org/abs/2604.26074).
- [12] Z. Wang, S. Mahar, L. Li, J. Park, J. Kim, T. Michailidis, Y. Pan, M. Shen, T. Rosing, D. Tullsen, S. Swanson, J. Zhao, The hitchhiker's guide to programming and optimizing cache coherent heterogeneous systems: CXL, NVLink-C2C, and AMD infinity fabric (2024). [arXiv:2411.02814](https://arxiv.org/abs/2411.02814).
- [13] R. Xie, A. U. Haq, Y. Fang, L. Ma, Z. B. Engineer, L. Liu, T. Zhang,

TRACE: Unlocking effective CXL bandwidth via lossless compression and precision scaling (2025). [arXiv:2509.03377](https://arxiv.org/abs/2509.03377).

- [14] Y. Wang, L. Wu, W. Hong, Y. Ou, Z. Wang, S. Gao, J. Zhang, S. Ma, D. Dong, X. Qi, M. Lai, N. Xiao, CXL-DMSim: A full-system CXL disaggregated memory simulator with comprehensive silicon validation, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [15] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22), USENIX Association, 2022.
URL <https://www.usenix.org/conference/osdi22/presentation/yu>
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, ACM, New York, NY, USA, 2023.
URL <https://arxiv.org/abs/2309.06180>
- [17] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.
- [18] W. Lee, J. Lee, J. Seo, J. Sim, InfiniGen: Efficient generative inference of large language models with dynamic KV cache management, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), USENIX Association, Santa Clara, CA, USA, 2024.
URL <https://www.usenix.org/conference/osdi24/presentation/lee-wonbeom>
- [19] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot, in: 23rd USENIX Conference on File and Storage Technologies (FAST '25), USENIX Association,

Santa Clara, CA, USA, 2025, pp. 155–170, best Paper Award.
URL <https://arxiv.org/abs/2407.00079>

- [20] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), USENIX Association, Santa Clara, CA, USA, 2024.
URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [21] J. Weiner, N. Agarwal, D. Schatzberg, L. Yang, H. Wang, B. Sanouillet, B. Sharma, T. Heo, M. Jain, C. Tang, D. Skarlatos, TMO: Transparent memory offloading in datacenters, in: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2022, pp. 609–621. doi:10.1145/3503222.3507731.
- [22] A. Jain, H. Lin, C. Villavieja, B. Kasikci, C. Kennelly, M. Hashemi, P. Ranganathan, Limoncello: Prefetchers for scale, in: R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, D. Tsafir (Eds.), Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024– 1 May 2024, ACM, 2024, pp. 577–590. doi:10.1145/3620666.3651373.
URL <https://doi.org/10.1145/3620666.3651373>
- [23] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101.
- [24] R. Chen, Z. Ding, S. Zheng, C. Zhang, J. Leng, X. Liu, Y. Liang, MAGIS: Memory optimization via coordinated graph transformation and scheduling for DNN, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '24, ACM, La Jolla, CA, USA, 2024, pp. 607–621. doi:10.1145/3620666.3651330.
URL <https://doi.org/10.1145/3620666.3651330>
- [25] P. Zhao, H. Zhang, F. Fu, X. Nie, Q. Liu, F. Yang, Y. Peng, D. Jiao, S. Li, J. Xue, Y. Tao, B. Cui, MEMO: Fine-grained tensor management

for ultra-long context LLM training, Proceedings of the ACM on Management of Data 3 (1) (2025). doi:10.1145/3709703.
URL <https://arxiv.org/abs/2407.12117>

- [26] S. Williams, A. Waterman, D. Patterson, Roofline: An insightful visual performance model for multicore architectures, Communications of the ACM 52 (4) (2009) 65–76.